

Prompt Engineering Best Practices

#consume

#AI

#book

this page include the rules to following to get the good prompt.

Write Clear

- Explain clearly with out **ambiguity**.
- Ask the model to adopt the **persona**
- Provide Example

	User prompt	AI response
Without example	Will Santa bring me presents on Christmas?	Santa Claus is a fictional character that kids believe in, bringing magic and excitement to Christmas with stories of his sleigh, reindeer, and gift-giving journey on Christmas Eve. Many parents encourage this belief, as it adds a layer of fun and wonder for children.
With example	Q: Is the tooth fairy real? A: Of course! Put your tooth under your pillow tonight. The tooth fairy might visit and leave you something. Q: Will Santa bring me presents on Christmas? A:	Yes, absolutely! Santa loves to bring presents to kids who believe in him and have been kind all year. Just make sure you're extra good, leave out some cookies, and you might wake up to find gifts under the tree on Christmas morning!

- Difference of given example or not.
- Then we can **reduce** amount of token with this technical is **Label** them.

Prompt	# tokens (GPT-4)
Label the following item as edible or inedible.	38
Input: chickpea	
Output: edible	
Input: box	
Output: inedible	
Input: pizza	
Output:	
Label the following item as edible or inedible.	27
chickpea --> edible	
box --> inedible	
pizza -->	

- Specific output format : Can used some special character to mark end of prompt,

Prompt	Model's output
Label the following item as edible or inedible.	tacos --> edib le
pineapple pizza --> edible	
cardboard --> inedible	
chicken	
Label the following item as edible or inedible.	edible
pineapple pizza --> edible	
cardboard --> inedible	
chicken -->	

Let model know the strcuture output.

⚠️ Specific marker should not a instructure in prompt because it make model confused.

Provide Sufficient Context

The reference text is necessary context or tool to given context to help model doing better. called **Context Construction** that a tool include **data retrieval**, and **websearch**.

Relevance this section ...

- [RAG](#)

Break Complex Tasks into Simpler Subtasks

- Extract a big of 1 prompt to subtasks.
- Increase more token

The main idea is ...

1. Intent classification : categorize a user prompt.
2. Generating response : base on intent, instruct the model how to response.

```
**Prompt 1 (intent classification)**  
  
**SYSTEM**  
You will be provided with customer service queries. Classify  
each query into  
a primary category and a secondary category. Provide your output  
in json  
format with the keys: primary and secondary.
```

Primary categories: Billing, Technical Support, Account Management, or General Inquiry.

Billing secondary categories:

- Unsubscribe or upgrade
- ...

Technical Support secondary categories:

- Troubleshooting
- ...

****USER****

I need to get my internet working again.

****Prompt 2 (response to a troubleshooting request)****

****SYSTEM****

You will be provided with customer service inquiries that require troubleshooting in a technical support context. Help the user by:

- Ask them to check that all cables to/from the router are connected. Note that it is common for cables to come loose over time.
- If all cables are connected and the issue persists, ask them which router model they are using.
- If the customer's issue persists after restarting the device and waiting 5 minutes, connect them to IT support by outputting {"IT support requested"}.
- If the user starts asking questions that are unrelated to this topic then confirm if they would like to end the current chat about troubleshooting and classify their request according to the following scheme:

```
<insert primary/secondary classification scheme from above here>

**USER**

I need to get my internet working again.
```

Give the more time to think

This section implement the CoT concept. CoT stand for chain of thought. In the prompt you can add the following word

- Thinking step by step
- Explain your decision
- follow these steps, then give the step to model.
- One-shot CoT: like a other method but given some example.

Original query	Which animal is faster: cats or dogs?
Zero-shot CoT	Which animal is faster: cats or dogs? Think step by step before arriving at an answer.
Zero-shot CoT	Which animal is faster: cats or dogs? Explain your rationale before giving an answer.
Zero-shot CoT	Which animal is faster: cats or dogs? Follow these steps to find an answer: 1. Determine the speed of the fastest dog breed. 2. Determine the speed of the fastest cat breed. 3. Determine which one is faster.
One-shot CoT (one example is included in the prompt)	Which animal is faster: sharks or dolphins? 1. The fastest shark breed is the shortfin mako shark, which can reach speeds around 74 km/h. 2. The fastest dolphin breed is the common dolphin, which can reach speeds around 60 km/h. 3. Conclusion: sharks are faster.

Iterate on Your Prompts

1. Model Quirks & Behavior Profiling : each model better differ field.
2. Systematic Prompt Versioning : try many prompt to know the model used

[Prompting Guide](#) provide by developer and store the prompt version and response.

3. System-Level Evaluation Metrics : used standard Evaluation Data and compare the performance each prompt that improve whole's system or worsen.

Evaluate Prompt Engineering Tools

Tools that provide input, ouput evaluation metric and evaluation data for your task :

- [OpenPrompt](#)
- [DSPy](#)
- Promptbreeder
- TextGrad

The core of these tools are given original prompt to other ai to build better prompt automated.

 **More AI more Bills. Those tools are can make bills growth exponential if your not checked it.**

Organize and Version Prompts

- Separate Prompt from code

```
file: prompts.py
GPT4o_ENTITY_EXTRACTION_PROMPT = [YOUR PROMPT]

file: application.py
from prompts import GPT4o_ENTITY_EXTRACTION_PROMPT
def query_openai(model_name, user_prompt):
    completion = client.chat.completions.create(
        model=model_name,
        messages=[
            {"role": "system", "content":
GPT4o_ENTITY_EXTRACTION_PROMPT},
            {"role": "user", "content": user_prompt}
        ]
    )
```

Pros:

- Reusability
- Testing
- Readability
- Collaboration

Prompt Meta data Structure for across multi-application, In python object.

```
from pydantic import BaseModel

class Prompt(BaseModel):
    model_name: str
    date_created: datetime
    prompt_text: str
    application: str
    creator: str
```

the prompt template should include Operational Configuration such as

- The model endpoint URL
- The ideal sampling parameters, like temperature or top-p
- The input schema
- The expected output schema (for structured outputs)

and you can create a [dot prompt file](#) format to store prompts, below is an example of a firebase .prompt file.

```
---
model: vertexai/gemini-1.5-flash
input:
  schema:
    theme: string
output:
  format: json
  schema:
    name: string
    price: integer
    ingredients(array): string
---
```


Generate a menu item that could be found at a {{theme}} themed restaurant.

You have 2 ways to store it.

1. Github repo : gather prompt in the once place.
2. Prompt Catagory : Store prompt version each application.

Relevance

- [Prompting Guide](#)
- [Defensive Prompt Engineering](#)